

Qwen3.5-0.8B训练

1. 项目背景及全流程梳理

腾讯云提出优化训练需求

要求多模态数据 seq len=1423 图片=1000×720 数据随机生成即可 训练框架accelerator+deepspeed zero1 预期性能10000+Tokens/sec

通过

- 调整deepspeed zero1为零0
- 增大mbs
- 使用flash-linear-attention替代flash attention

三种方法实现大幅度优化

1.1 accelerate + deepspeed

Accelerate是轻量级分布式训练抽象层（这里是deepspeed） 统一训练框架后端 一键开启混合混合精度训练 并统一处理model/dataloader/optimizer/scheduler等 本身并不参与具体的训练任务 只是封装分布式逻辑

代码块

```
1 def main(args):
2     set_seed(args.seed)
3     accelerator = Accelerator()
4
5     model = setup_model(args.model_path,
6                          attn_implementation=args.attn_implementation)
7     processor = setup_processor(args.model_path)
8     collate_fn = make_collate_fn(processor, args.max_seq_length)
9
10    world_size = accelerator.num_processes
11    micro_batch_size =
12    accelerator.deepspeed_plugin.deepspeed_config["train_micro_batch_size_per_gpu"]
13
14    base_dataset = SimpleDataset(args.data_path)
```

```

13 min_samples = micro_batch_size * world_size
14 k = (min_samples + len(base_dataset) - 1) // len(base_dataset)
15 dataset = ConcatDataset([base_dataset] * k) if k > 1 else base_dataset

```

代码块

```

1 optimizer = DummyOptim(model.parameters())
2 scheduler = DummyScheduler(optimizer)
3 model, optimizer, scheduler = accelerator.prepare(model, optimizer,
4 scheduler)
5 model.train()

```

所以可这里可以理解成Accelerate 管理分布式环境 DeepSpeed 提供训练/优化引擎

Deepspeed框架有不同的逻辑 之前周报里也整理过不同stage的切分对象

成都蓉光信息技术有限公司 > 客户应用优化部 >  团队会议 > 团队周会 > 战略客户支持组 > week20(0511~0515) > 张一凡-W20-0515 平

最近修改: 5月16日 16:01

 分享

<<

张一凡-W20-0515

1. 项目进展

1.1 科大讯飞&合肥大数据公司-Deepseek V4 ...

1.1.1 本周我做了什么

1.1.2 下周我计划做什么

1.1.3 卡点 / 需要 Lead 协助的事项

1.2 腾讯云-yiyao项目-基于deepspeed框...

1.2.1 本周我做了什么

1.2.2 下周我计划做什么

1.2.3 卡点 / 需要 Lead 协助的事项

1.3 Minimax-M2.7-bf16部署

1.3.1 本周我做了什么

1.3.2 下周我计划做什么

• Deepspeed



和Megatron相似之处在于 Deepspeed的核心思想也在于切分 但是前者切分的是模型参数 后者切分的是训练中的并行数据的状态 如优化器状态 梯度等 切分到不同的DCU上 这个最核心的创新叫做ZeRO (零冗余优化器) Zero也分为三种

Zero1: 切分优化器状态 Zero2:切分优化器状态和梯度 这俩多卡配合时可训练更大模型 (因为每卡只存1/N的优化器状态)

Zero3: 切分优化器状态、梯度和模型参数 在前向/反向时需要临时all-gather重建完整参数

Zero1和Zero2不切分模型权重 前提是单卡显存够放下模型 切分一下优化器和梯度罢了 上面提到的deepspeed训练任务就是基于zero2实现的 然后Zero3的策略就比较强了 是在两个层面进行切分 第一个层面的切分对象是模型参数 然后在此基础上切分训练中的数据 除此之外还有ZeRO-Offload 和 ZeRO-Infinity 技术可以将一部分数据“卸载”到CPU内存上进一步突破了单GPU的显存限制

客户一开始用的stage1 切分的是优化器状态 这样的话每张卡仅保存1/N 的优化器参数 在优化器更新时有额外的All-Gather 带来额外的通信开销 因为模型小 就算不切分也能放得下 所以这种开销是完全可以避免的 换句话说压根不需要 因此更换stage1为stage0 后者倒是头一次听说

根据zero1到3切分对象越来越多的规律来看 反推zero0就是什么都不切 所有卡保留完整参数 通信角度来说就是前向传播时直接使用完整通信 方向传播时使用完整参数 方向传播时allreduce一波梯度 参数也是完整更新的 所以尽管这个模型单卡能放下 单卡的时候stage0和1实际上是一样的 但是客户要求8卡 所以还是zero0比较好

既然zero0什么都不切分 没有发挥出zero123省显存的优势 那么在这里的作用是什么呢

代码块

```

1  {
2      "bf16": { "enabled": true },
3      "optimizer": {
4          "type": "AdamW",
5          "params": { "lr": 2e-5, "betas": [0.9, 0.95], "weight_decay": 0.1 }
6      },
7      "scheduler": {
8          "type": "WarmupLR",
9          "params": { "warmup_min_lr": 2e-5, "warmup_max_lr": 2e-5,
10             "warmup_num_steps": 1 }
11      },
12      "zero_optimization": { "stage": 0 },
13      "gradient_clipping": 1.0,
14      "train_batch_size": "auto",
15      "train_micro_batch_size_per_gpu": 2,
16      "gradient_accumulation_steps": 8,
17      "steps_per_print": 9999
18  }

```

根据zero0的配置文件可知 他可以自动计算全局bs 无需手动在代码中实现梯度累积逻辑 不过这里的优化器用的是torch原生的AdamW 这说明优化器更新在cpu上完成 每次同步还得有cpu-dcu-cpu的数据搬运 可以换成fused_AdamW 这样的话所有的操作都在dcu上完成 性能还可进一步提升 不过根据trace文件 优化器占比并不大 所以这部分更换收益也不会很大

除了优化器以外 stage0还自动在反向传播后、优化器更新前执行梯度裁剪 避免梯度爆炸出现inf 并且还有学习率调度器集成

总结一下就是zero0虽然不切分任何对象 但提供优化器融合+梯度管理等能力 适合模型较小但需要优化训练效率的场景 acc+ds的配置是流水线式的串行任务流 acc作为管理层调用ds执行层 而 optimizer/dummy/调度器是占位符 acc 在prepare()时用 DeepSpeed 实现替换 这就允许同一份代码在不同后端（DDP/FSDP/DeepSpeed）间切换 十分方便

1.2 增大gbs

$gbs = mbs \times \text{gradient_accumulation_steps} \times \text{num_gpus}$ 增加gbs可以将多个小 batch 的计算合并成一次大 batch 的参数更新 增大了梯度累计 FusedAdam、梯度裁剪、参数 copy等操作多个micro-step才执行一次 减少了参数更新相关的操作频率 也就减少了相应的开销

1.3 flash linear attention

引入这个库的目的是优化Gated DeltaNet的计算 因为该模型架构并不是纯基于传统的transformer 除了标准的gated attention层外 还有Gated DeltaNet线性注意力层 二者比例为3:1 前者用于捕捉复杂的上下文关联 后者用于高效处理长序列

1.3.1 Flash attention for Gated Attention

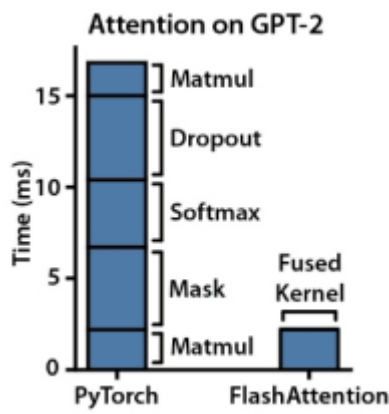
简单回顾一下 对于传统的注意力 fa是不改变算法本身 而是一种加速的方法 加速的核心思想是Tiling分块 + Online Softmax

- 分块：FlashAttention 把 Q、K、V 切分成小块 不在显存中完整保存注意力分数矩阵 而是分批将小块数据加载进 DCU缓存 循环完成计算 然后依靠Online Softmax 增量算法跨块维护 softmax 状态（最大值m、指数累加和d） 动态修正归一化权重最后拼接结果

公式:
$$\text{softmax}(x_i) = \frac{e^{x_i - m_j}}{\sum_{j=1}^N e^{x_j - m_j}} = \frac{e^{x_i - m}}{d}$$

Online Softmax 算法与传统的一样 自变量为增量而非全局

- 硬件层面来说 SRAM 带宽远高于 HBM 显存 容量很小 HBM 容量大但是带宽更低 FlashAttention 尽可能把计算放在 SRAM 中 减少 HBM 读写次数 从而实现传统注意力的加速计算 并且降低显存压力
- 除了分块 fa还是还是IO感知算法 根据存储层级DRAM内存→HBM 显存→片上 SRAM缓存的带宽和容量特性选择合适 Block 大小 最小化 HBM 读写次数 并且在此过程中通过Fused Kernel消除大量中间张量落地 如下图所示



在GPT2模型中原生attention与借助fa实现的耗时对比

1.3.2 Flash linear attention for DeltaNet

线性注意力；

$$\text{LinearAttention}(Q, K, V) = \phi(Q) \cdot (\phi(K)^T V)$$

ϕ 是一个核函数（如 $\text{elu}(x)+1$ 或 $\text{relu}(x)\text{relu}(x)$ ）

传统注意力是先算QK再算QKV 线性注意力先算KV 即 $d \times d$ 矩阵（ d 是特征维度）再左乘Q fla可以理解为线性注意力领域的fa 而DeltaNet是线性注意力领域的“升级”算法 传统的线性注意力的记忆方式是静态累积 不断累加新的信息 操作可以概括为状态 = 状态 + 新信息

而DeltaNet则是动态更新 会根据预测误差修正已有记忆 即状态 = 状态 - 旧预测 + 新目标 而fla作为线性注意力领域的fa 优化的基本思想是一致的 也遵循了FA开创的“IO感知”优化范式 核心策略也是分块计算和融合算子 唯一的不同就是优化的对象 fa的不再赘述 fla的优化对象是线性RNN 如DeltaNet、Mamba、GLA等 为了优化线性RNN的递归计算 fla采用了一套结合了并行和循环的混合策略

- 分块并行：将长序列切分成许多小块 并在每个小块内部并行计算所有时间步的状态 从而摆脱完全递归带来的性能瓶颈 但是为了并行计算一个小块内部的状态 需要预知这个小块在块起始时的“初始状态” 因此很多中间状态必须被临时保存在显存中 这会带来额外的内存消耗和IO开销
- Tiled Flash Linear Attention (TFLA)：为了彻底解决这个问题 在FLA基础上又提出了TFLA（平铺闪存线性注意力） 它在分块的基础上 在块内部又增加了一层平铺（Tiling） 使得算法能够使用任意大的分块 并且通过精妙的计算调度 实现了更高的算术强度 进一步提高DCU的使用率

fla的实现：包直接pip install 社区版的即可 具体实现位置：

在/usr/local/lib/python3.10/dist-

packages/transformers/models/qwen3_5/modeling_qwen3_5.py开头

代码块

```
1  if is_flash_linear_attention_available():
2      from fla.modules import FusedRMSNormGated
3      from fla.ops.gated_delta_rule import chunk_gated_delta_rule,
        fused_recurrent_gated_delta_rule
4  else:
5      chunk_gated_delta_rule, fused_recurrent_gated_delta_rule = None, None
6      FusedRMSNormGated = None
```